A Comparison of Heuristic and Metaheuristic Methods for solving the Traveling Salesman Problem

1st Daniel Schafhäutle
Computational and Data Science
FHGR
Chur, Switzerland
daniel.schafhautle@stud.fhgr.ch

2nd Joshua Kohler
Computational and Data Science
FHGR
Chur, Switzerland
joshua.kohler@stud.fhgr.ch

3rd Gian-Luca Lanfranchi
Computational and Data Science
FHGR
Chur, Switzerland
gianluca.lanfranchi@stud.fhgr.ch

***Abstract*—The traveling salesman problem (TSP) is an NP hard combinatorial optimization problem, that asks for the shortest possible route that visits a set of cities and returns to the starting city. Solving TSPs exactly using brute-force methods quickly becomes computationally infeasible as the number of cities increases. However, there exists a number of heuristic and metaheuristic algorithms, that can find reasonably good solutions in a much shorter time. This paper presents a comparison of five such methods, namely: Simulated Annealing, Tabu Search, 2-Opt, Iterated Local Search and a Genetic Algorithm. Algorithms are evaluated using synthetic benchmark TSP instances of sizes n = [10, 25, 50, 100, 200, 500, 800, 1000, 1500, 2000] with k = 30 instances each. We preform this experiment with two types of instances: one with uniformly sampled cities and one with clustered cities. They are then compared based on the solution quality measured as the gap to the optimal solution obtained by the Concorde TSP solver, and the time required to find the solution. The goal of this paper is to compare the strengths and weaknesses of these algorithms, that are the basis of many state of the art methods for solving TSPs and provide a starting point for further research into this area.**

I. Introduction

The traveling salesman problem (TSP) is a classic combinatorial optimization problem that asks for the shortest possible route that visits a set of cities and returns to the origin city. More formally, given a complete Graph $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_n\}$ is the set of nodes representing the cities, and $d(v_i, v_j)$ is the distance associated with the edge between each pair of nodes $v_i, v_j \in V$, then the solution to a TSP is the ordering of cities $(v_{\pi(1)}, v_{\pi(2)}, \ldots, v_{\pi(n)})$ that minimizes the total tour length:

$$\left(\sum_{i=1}^{n-1} d(v_{\pi(i)}, v_{\pi(i+1)})\right) + d(v_{\pi(n)}, v_{\pi(1)}) \tag{1}$$

Where $\pi(i)$ determines the index of the city that is visited at step i [1].

This is an NP-hard combinatorial optimization problem. That means, that there is no known polynomial time algorithm, that can solve it optimally for all instances [1]. It is so hard to solve, because the number of possible tours is $(n-1)!/2$ and grows with $\mathcal{O}(n!)$ [1]. This makes it infeasible to brute-force a solution for problems with more than a few nodes. A problem with 100 nodes has 10^{155} possible tours, to check all of them naively would take longer than the age of the universe. Because we don't gave that kind of time, we need to find ways to quickly find solutions to TSPs that are *good enough*, meaning that they are not optimal, but sufficiently close to the optimal solution. However, for a lot of applications a solution that is not quite optimal is all that is needed. Heuristics are used to find these near optimal solutions within reasonable time.

II. Related Work

The TSP is one of the most intensively studied combinatorial optimization problems. Its NP-hard nature [1] makes exact solution methods infeasible for large instances, yet the search for optimal solutions has driven significant progress in both exact and heuristic algorithms for solving the TSP but also in the area of combinatorial optimization as a whole.

A. Exact Solvers

Exact solutions to the TSP can be obtained by modelling it as a dynamic programming or integer linear programming problem and then solving it using branch-and-bound or branch-and-cut methods [2]. The gold standard for exact solvers of the TSP is Concorde [3], it has been developed and optimized over years specifically to solve TSPs and was able to obtain an optimal tour for a problem with 85900 cities, computing it required over 136 CPU years [3]. While exact solvers are the gold standard for verifying solution quality, their runtime increases exponentially, limiting their applicability to large instances.

B. Heuristics and Metaheuristics for the TSP

To overcome the limitations of exact methods, researchers have developed a wide range of heuristic and metaheuristic algorithms that can find near-optimal solutions in reasonable time.

Heuristics for the TSP can be separated into two groups of algorithms: *tour construction procedures*, which incrementally build a tour by adding a new node to the tour at each step, and *tour improvement procedures*, which start from an initial tour and then try to improve that tour by making changes. *Composite procedures* are algorithms, that start with a solution generated by a construction procedure and then improve it with an improvement procedure [4].

Metaheuristics (2-Opt) are algorithms that take into account the specific nature of the problem (TSP) and use that "meta"-knowledge to construct a more efficient algorithm. *Heuristics* (Simulated Annealing, Tabu Search, Iterated Local Search, Genetic algoritmhs) on the other hand are problem agnostic and can be applied to any problem that fits its requirements.

C. History

Among the most influential metaheuristics, *Simulated Annealing* (SA) introduced the idea of probabilistic acceptance of worsening moves to escape local optima.

Tabu Search (TS), originally proposed by Glover [4] and later refined by Gendreau and Potvin [5], uses explicit memory structures (tabu lists) to prevent cycling and guide the search.

Iterated Local Search (ILS) [6] builds chains of local optima by perturbing solutions between applications of a local search, achieving state-of-the-art performance on many problems.

Genetic Algorithms (GAs) for the TSP are reviewed by Larrañaga et al. [7], and more recent developments, including the EAX crossover that can find optimal tours for instances with up to 200,000 cities, are discussed by Whitley [8].

A more in-depth review of metaheuristics and heuristics applied to TSP is provided by Gendreau and Potvin [9], where they present a wide variety of both basic and advanced algorithms.

III. Materials and Methods

In this chapter several algorithms are introduced that are at the basis of many state of the art methods for solving the TSP. The goal is to introduce the reader to a variety of different techniques and concepts that are commonly used.

A. 2-Opt

2-Opt is one of the simplest and most widely used local search heuristics for the TSP [4]. Starting from some initial tour, it repeatedly removes two non-adjacent edges and reconnects the resulting two paths in the only other possible way, which is the same as reversing the segment between the two cut points. A move is accepted whenever it strictly decreases the tour length. The neighborhood of a tour with n cities therefore contains $\mathcal{O}(n^2)$ candidate moves. The search terminates in a local minimum, that is, a tour for which no single 2-Opt swap gives an improvement.

In this study, the initial tour is constructed by a deterministic nearest-neighbor heuristic (seeded for reproducibility): starting from a fixed city, the next city is always chosen as the closest unvisited one until all cities are included. The 2-Opt search itself uses a *first-improvement* strategy: as soon as an improving swap (i, j) is found, it is applied immediately and the scan is restarted from the beginning of the tour. This is repeated until a full pass over all pairs (i, j) finds no improving move, at which point the tour is 2-optimal.

Algorithm 1 2-Opt

```
s = GreedyInitialTour()
improved ← true
while improved do
    improved ← false
    for i = 0 to n − 3 do
        for j = i + 2 to n − 1 do
            a, b ← s_i, s_{i+1}
            c, d ← s_j, s_{(j+1) mod n}
            if d(a, c) + d(b, d) < d(a, b) + d(c, d) then
                reverse s_{i+1...j}
                improved ← true
                break out of both loops
            end if
        end for
    end for
end while
return s
```

The improvement condition compares the cost of the two edges that would be removed, $d(a, b) + d(c, d)$, with the cost of the two edges that would replace them, $d(a, c) + d(b, d)$. Because the segment between the two cut points is simply reversed, no other edge weights change and the cost difference can be evaluated in constant time per candidate move. 2-Opt is used here both as a single solver and as the embedded *LocalSearch* of the Iterated Local Search algorithm described below.

B. Simulated Annealing

Plain local search like 2-Opt always stops at the first local minimum it reaches, because it never accepts a move that makes the tour worse. Simulated Annealing fixes this by sometimes accepting worse tours on purpose, so that the search can escape a local minimum and keep looking for a better solution [4].

How willing the algorithm is to accept a worse tour is controlled by a single number called *temperature* T. At each step a neighbor tour is created with one random 2-Opt move, and the change in tour length $\Delta = \text{Cost(next)} - \text{Cost(current)}$ is computed. A better tour ($\Delta < 0$) is always accepted. A worse tour is accepted only with probability $e^{-\Delta/T}$. When T is high this probability is close to one, so many worse tours are accepted. When T is low it is close to zero, so the algorithm behaves almost like normal 2-Opt local search.

The temperature is lowered after every step with a geometric cooling schedule, $T(t) = T_0 \cdot 0.9999^t$. The starting temperature T_0 is not fixed but estimated for each problem so that the algorithm fits the scale of the distances: a few hundred random 2-Opt moves are sampled, the average of the worsening moves $\overline{\Delta}$ (mean delta) is taken, and T_0 is set so that

about 60% of such moves would be accepted at the start, i.e. $T_0 = \overline{\Delta}/\ln(1/0.6)$. The search starts from a greedy nearest-neighbor tour and runs for a fixed budget of 100 000 iterations (or stops early once the temperature is practically zero), always returning the best tour seen so far.

Algorithm 2 Simulated Annealing

$T_0 = \text{EstimateInitialTemperature}()$
current = GreedyInitialTour()
best = *current*
for $t = 1$ **to** MaxIterations **do**
  $T = T_0 \cdot 0.9999^t$
  if $T \approx 0$ **then**
    return *best*
  end if
  next = Random2OptMove(*current*)
  $\Delta = \text{Cost}(next) - \text{Cost}(current)$
  if $\Delta < 0$ **then**
    current = *next*
    if Cost(*current*) < Cost(*best*) **then**
      best = *current*
    end if
  else
    accept *current* = *next* with probability $e^{-\Delta/T}$
  end if
end for
return *best*

A random 2-Opt move is the same edge swap as in normal 2-Opt, but here the two cut points are chosen at random instead of by scanning for an improvement. This keeps each step cheap while still letting the temperature decide whether a worse tour is kept or thrown away.

C. Tabu Search

Tabu Search (TS) extends classical local search by using memory structures to escape local minima and to avoid cycling [5]. While a plain local search like 2-Opt stops at the first local optimum, TS continues from there by accepting non-improving moves. To prevent the search from immediately undoing such a move and returning to the previous solution, it keeps a *tabu list* of recently performed moves. Moves that are *tabu* (forbidden) are not allowed unless they satisfy an *aspiration criterion* (e.g., they lead to a new global best).

In the algorithm implementation 3, the search space is the set of all Hamiltonian tours. The neighborhood is defined by all 2-Opt swaps, giving $\mathcal{O}(n^2)$ candidate moves per iteration. The initial solution is constructed by the same greedy nearest-neighbor heuristic used for 2-Opt and Simulated Annealing.

At each iteration, the algorithm examines the whole 2-Opt neighborhood of the current tour. For every pair (i, j), it generates a neighbor by reversing the segment $i + 1 \dots j$ and computes its total length. A move is considered admissible if either it is not tabu, or its tour length is better than the best known solution (aspiration criterion). Among all admissible moves, the one with the smallest tour length is selected (*best improvement* strategy).

Algorithm 3 Tabu Search

$s \leftarrow$ GreedyInitialTour() {without closing edge}
$s_{\text{best}} \leftarrow s$
tabuList $\leftarrow$ empty queue
for iteration $= 1$ **to** maxIterations **do**
  bestNeighbor $\leftarrow$ None
  bestMove $\leftarrow$ None
  for each 2-Opt move (i, j) in the current tour **do**
    $s' \leftarrow$ tour after applying move (i, j)
    isTabu $\leftarrow (i, j) \in$ tabuList
    if not isTabu **or** Cost(s') < Cost(s_{best}) **then**
      if Cost(s') < Cost(bestNeighbor) **then**
        bestNeighbor $\leftarrow s'$
        bestMove $\leftarrow (i, j)$
      end if
    end if
  end for
  if bestNeighbor = None **then**
    break
  end if
  $s \leftarrow$ bestNeighbor
  add bestMove to tabuList
  if len(tabuList) > tenure **then**
    remove oldest entry
  end if
  if Cost(s) < Cost(s_{best}) **then**
    $s_{\text{best}} \leftarrow s$
  end if
end for
return s_{best} with closing edge

The selected move is then applied to become the new current tour, and the move (represented as the pair (i, j)) is added to the tabu list. If the tabu list exceeds a given *tabu tenure*, the oldest entry is removed. The tabu tenure is set to $\max(7, \lfloor\sqrt{n}\rfloor)$. This choice follows a common static rule depending on the problem size, where $\sqrt(n)$ provides a balanced trade-off between intensification (not overly restricting the search) and diversification (preventing cycles) [10] cited this, but it's behind a paywall [11]

The search continues for a fixed number of iterations, which depends on the problem size:
MAX_ITERATIONS $= \max(50, 5000/n)$

This scaling ensures that large instances are not overwhelmed by an excessive number of iterations, while small instances still receive enough exploration. The algorithm returns the best tour found over the whole search.

The aspiration criterion used here is the simplest and most common one. A tabu move is allowed if it produces a tour strictly shorter than the global best tour found so far [5, Sec. 2.3.5]. This guarantees that no improving move that would set a new record is ever rejected. The tabu list stores only the pair (i, j) representing the inversion endpoints. This prevents the

immediate reversal of that particular 2-Opt swap, but does not forbid other moves that involve the same edges or cities, thus keeping the search sufficiently flexible.

Compared to Simulated Annealing, Tabu Search uses a deterministic move selection rule (best admissible neighbor) and relies on explicit memory to avoid cycles rather than on probabilistic acceptance. For the TSP, TS often finds very good solutions in a moderate number of iterations, because its best-improvement strategy aggressively descends toward local optima, while the tabu list provides just enough diversification to escape them. The implementation follows the classic template described in [5, Sec. 2.3.6].

D. Iterated Local Search

The idea of iterated local search is that instead of searching the whole space of all candidate solutions, we focus only on candidate solutions in the local neighborhood of an initial solution. These are usually generated by some kind of local search heuristic. It is conceptually simple, but has led to state-of-the-art algorithms for many problems [6]. In its essence, iterated local search builds a chain of solutions by exploring the currently best solutions local neighborhood but introduces some randomness in order to not get stuck in local minima.

Algorithm 4 Iterated Local Search

$s_0 = \text{GenerateInitialSolution}()$
$s^* = \text{LocalSearch}(s_0)$
$s_{\text{best}} = s^*$
repeat
  $s' = \text{Perturbation}(s^*)$
  $s^{*\prime} = \text{LocalSearch}(s')$
  if $\text{Cost}(s^{*\prime}) < \text{Cost}(s_{\text{best}})$ **then**
    $s_{\text{best}} = s^{*\prime}$
  end if
  if $\text{Cost}(s^{*\prime}) < \text{Cost}(s^*)$ **then**
    $s^* = s^{*\prime}$
  end if
until termination condition met (e.g., iterations $= 200$)
return s_{best}

Iterated local search can be used with any heuristic optimization algorithm (*LocalSearch*) that takes in a tour and returns one that is at least as good as the input. It starts from some kind of initial solution s_0 that is improved to s^* by one iteration of the *LocalSearch* algorithm. This is the current solution. Then we try to improve s^* for a number of iterations or until a termination condition is met.

For the algorithm to be able to escape the local neighborhood, instead of applying *LocalSearch* directly to the current solution s^* again, first a change or *Perturbation* is applied that leads to an intermediate solution s'. Then *LocalSearch* is applied to s' and we get a new candidate solution $s^{*\prime\prime}$. If this candidate solution $s^{*\prime\prime}$ has a better *Cost* than the current one, it becomes the new current solution $s^{*\prime}$. If it is better than the current best, it also becomes the new current best s_{best}.

The *Cost* is the function that is to be minimized by the algorithm. In the case of the TSP that is the length of the tour.

The *Perturbation* is the mechanism that allows the algorithm to escape local minima. This function should not be reversible by the *LocalSearch*. So if the algorithm steps from $s_1^* \to s_2^*$ it then should be unable to do $s_2^* \to s_1^*$ in the same iteration.

Usual choices, and what was also used in this study are 2-Opt for the *LocalSearch* and 4-Opt for the *Perturbation*, this is a good choice because a 4-Opt move cannot be undone by a 2-Opt move and thereby return to the last starting point of the current iteration.

4-Opt (also called double-bridge move) randomly splits the tour into four segments $A + B + C + D$ and reconnects them as $A + C + B + D$ [6].

E. Genetic Algorithm

Genetic Algorithms (GAs) are population-based metaheuristics inspired by natural evolution [8]. A population of candidate tours evolves over several generations using selection, crossover, and mutation. The algorithm maintains a diverse set of solutions, which helps to explore the search space more broadly than local search alone.

1) Representation and Initialization: Each tour is represented as a permutation of the n city indices (without the closing edge). The initial population consists of POPULATION_SIZE $= 100$ individuals. One individual is created by the same greedy nearest-neighbor heuristic used for the other algorithms. The remaining 99 individuals are random permutations of the cities. This hybrid initialization ensures that a good starting solution is always present while providing diversity.

Algorithm 5 Genetic: Initialization of the Population

$n \leftarrow$ number of cities
popSize $\leftarrow 100$
population $\leftarrow$ empty list
greedy $\leftarrow$ GreedyInitialTour() {without closing edge}
population.append(greedy)
while len(population) $<$ popSize **do**
  randomTour $\leftarrow$ random permutation of $\{0, \ldots, n-1\}$
  population.append(randomTour)
end while
return population

2) Selection and Elitism: At each generation, the next population is built as follows. First, the ELITISM_COUNT $= 2$ best tours of the current population are copied unchanged into the new population (*elitism*), guaranteeing that the best solution never degrades. The remaining slots are filled by repeatedly selecting two parents using *tournament selection*: Pick TOURNAMENT_SIZE $= 5$ random individuals from the population and return the one with the shortest tour length. This is done twice to obtain two parents. Tournament selection applies selection pressure without requiring global sorting.

Algorithm 6 Genetic: Tournament Selection

Require: population, costs, tournamentSize
indices ← random choice of tournamentSize
↪indices from population
bestIdx ← index in indices with minimum cost
return population[bestIdx]

3) Crossover and Mutation: With probability CROSSOVER_RATE $= 0.9$, the two parents are recombined using *Order Crossover (OX)*. OX works by:

1) choosing two random cut points $i < j$ in the parent permutation,
2) copying the segment between i and j from the first parent into the child at the same positions,
3) filling the remaining positions in the child with the cities from the second parent in the order they appear, skipping those already placed.

Algorithm 7 Genetic: Order Crossover (OX)

Require: parent1, parent2 (permutations of $0, \ldots, n-1$)
$n \leftarrow$ len(parent1)
$i, j \leftarrow$ uniformly random positions with $i \leq j$
child $\leftarrow$ [ ] with length n
child$[i:j] \leftarrow$ parent1$[i:j]$
fill $\leftarrow$ cities in parent2 not in child$[i:j]$
$k \leftarrow n - j - 1$ {number of positions after j}
child$[j+1:n] \leftarrow$ fill$[0:k]$ preserving cyclic order
child$[0:i-1] \leftarrow$ fill$[k+1:]$
return child

This operator preserves the relative order of cities from both parents and produces a valid tour. If crossover is not applied (with probability 0.1), the child is an exact copy of the first parent.

After crossover, each child is mutated with probability MUTATION_RATE $= 1/n$. The mutation operator is *inversion*: Two random positions $i < j$ are chosen, and the segment between them is reversed. Inversion is a common mutation for permutation representations because it preserves the set of edges (i.e., does not introduce duplicate cities).

4) Generations and Termination: The number of generations is set to $\max(100, 10 \cdot n)$, where n is the number of cities. This scaling gives more generations to larger problems, allowing the GA to converge. After each generation, the best tour in the new population is compared with the global best found so far. The global best is updated if an improvement is found.

The algorithm returns the global best tour after the final generation.

Algorithm 8 Genetic: Inversion Mutation

Require: tour (list of cities)
$n \leftarrow$ len(tour)
$i, j \leftarrow$ two distinct random positions in $[0, n-1]$, with $i \leq j$
tour$[i:j+1] \leftarrow$ reversed(tour$[i:j+1]$)
return tour

The genetic algorithm does not use any local improvement. It relies entirely on crossover and mutation to evolve good tours. Despite this, the combination of a greedy initial solution, elitism, and the order crossover operator often yields competitive results, especially for smaller instances. For larger problems, the GA can be slower than local search methods, but it maintains a broad exploration of the search space.

Algorithm 9 Genetic Algorithm

population $\leftarrow$ InitializePopulation() {1 greedy + 99 random}
$s_{\text{best}} \leftarrow$ best tour in population
for generation $= 1$ **to** maxGenerations **do**
  newPopulation $\leftarrow$ elite individuals from population
  while newPopulation not full **do**
    $p_1 \leftarrow$ TournamentSelect(population)
    $p_2 \leftarrow$ TournamentSelect(population)
    if random float $\in [0,1] <$ crossoverRate **then**
      child $\leftarrow$ OrderCrossover(p_1, p_2)
    else
      child $\leftarrow p_1$
    end if
    if random float $\in [0,1] <$ mutationRate **then**
      child $\leftarrow$ InversionMutate(child)
    end if
    add child to newPopulation
  end while
  population $\leftarrow$ newPopulation
  $s_{\text{best}} \leftarrow \min(s_{\text{best}},$ best in population$)$
end for
return s_{best} with closing edge

An advanced discussion of genetic algorithms for combinatorial optimization, including Gray-box optimization and the EAX crossover, is given in [8].

IV. Setup

All experiments were conducted on a machine Please adjust this value with the computer which has produced the results with an 12-core AMD Ryzen 9 5900X CPU, 32-GB of RAM, and a 1-TB NVMe SSD running Ubuntu 22.04 LTS. The implementation and all subsequent analyses were carried out in Python 3.11.13.

A. Software Environment and Dependencies

The entire codebase is written in Python and relies on three core libraries: nosep

- `numpy` for numerical operations

- `matplotlib` for plotting
- `pandas` for data handling.

B. Reproducibility Measures

To guarantee that the experiments are fully reproducible, the following measures were taken:

nosep

- A fixed random seed (`np.random.seed(42)`) is used in the data generation script and in every solver that relies on pseudo-random numbers (e.g., random 2-Opt moves, tournament selection, mutation). This ensures that all stochastic decisions are deterministic across runs.
- The exact version of each dependency is pinned in `pyproject.toml`, and the environment is managed by `uv`.
- The complete source code is included in the supplementary material, and the command lines for generating instances, running the benchmark, and producing plots are documented in the README.

C. Running the Benchmark

The benchmark is executed via the script `src/benchmark.py`. It automatically discovers all problem instances in the `data/` directory and runs the selected solvers in parallel using Python's `ProcessPoolExecutor`. The number of parallel workers can be controlled with the `--workers` flag (defaults to the number of CPU cores). Already computed results are skipped to allow incremental runs.

All results are appended to `results/results.json` and also exported to `results/results.csv` for easier analysis. The base solver class automatically records the algorithm name, problem path, problem size, instance type, tour cost, solving time, a validity flag, and a timestamp.

D. Obtaining Optimal References

The optimal tour lengths (or best known lower bounds) are obtained using the CONCORDE TSP solver. Concorde is a widely used exact solver that implements cutting-plane methods and branch-and-bound. The binary was downloaded from the official Concorde website, placed in `bin/`, and made executable. The wrapper `src/algorithms/concorde_solver.py` invokes Concorde on each instance and parses its output. The resulting optimal costs serve as the ground truth for computing optimality gaps.

E. Post-processing and Plotting

After all benchmark runs, aggregated statistics (mean and standard deviation of tour cost, solving time, and optimality gap per algorithm, problem size, and instance type) are computed by `src/utils/compute_statistics.py`. The plots shown in Section V (time scaling, solution quality, gap bars, and heatmaps) are generated by `src/utils/create_aggregate_plots.py`.

F. Dataset Generation

To evaluate the performance of the TSP algorithms a synthetic benchmark dataset of Euclidean TSP instances is generated. The generation process is controlled by a Python script that creates two types of point distributions: *uniform* and *clustered*. The script uses a fixed random seed (`42`) to ensure reproducibility.

1) Instance sizes and repetitions: For each problem size $n \in \{10, 25, 50, 100, 200, 500\}$ 30 independent instances were generated. The same set of instances is used for all algorithms, allowing a fair comparison.

2) Uniform instances: Uniform instances are created by drawing n coordinates independently from a uniform distribution over the unit square $[0, 1]^2$. The Euclidean distance between every pair of coordinates is computed to form a symmetric distance matrix.

3) Clustered instances: Clustered instances are designed to mimic real-world scenarios where coordinates of e.g. cities or harbors form clusters. The number of clusters is set to $\max(1, n//10)$. Cluster centers are sampled uniformly from $[0.1, 0.9]^2$ (avoiding the extreme borders). Each node is assigned to a center in a round-robin fashion and its coordinates are drawn from a normal distribution around that center with a standard deviation of 0.05. The resulting coordinates are clipped to the unit square to keep all points inside $[0, 1]^2$. The distance matrix is then computed as for the uniform case.

V. RESULTS

1) Benchmark: Mention greedy baseline

2) Hardware / Environment:

- Performance Metrics
- Plots

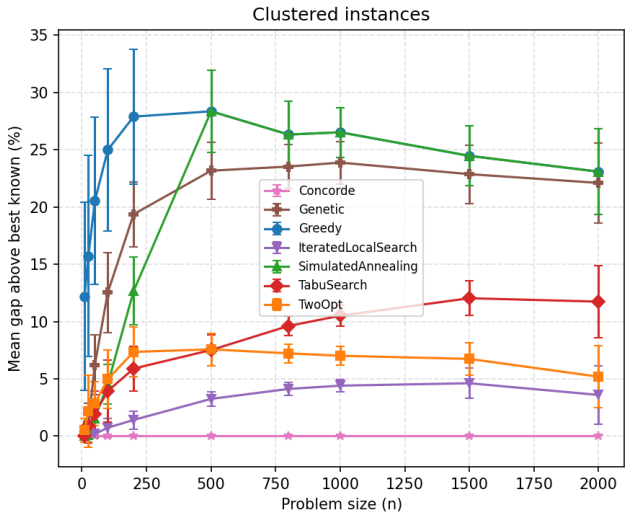

Fig. 1. Optimality gap of clustered algorithms for different sizes and distributions

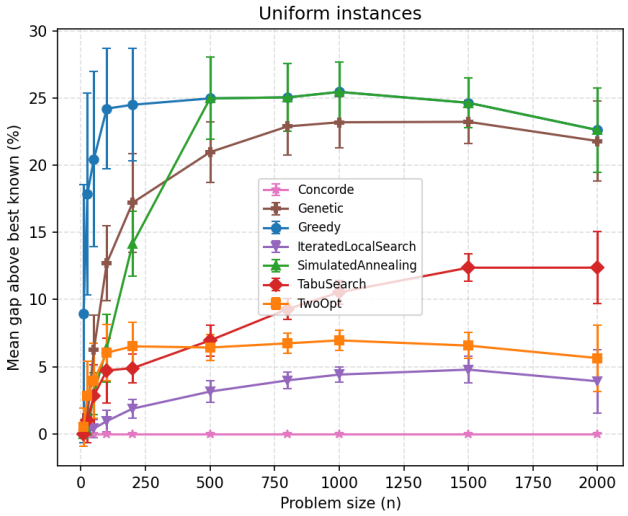

Fig. 2. Optimality gap of uniformed algorithms for different sizes and distributions

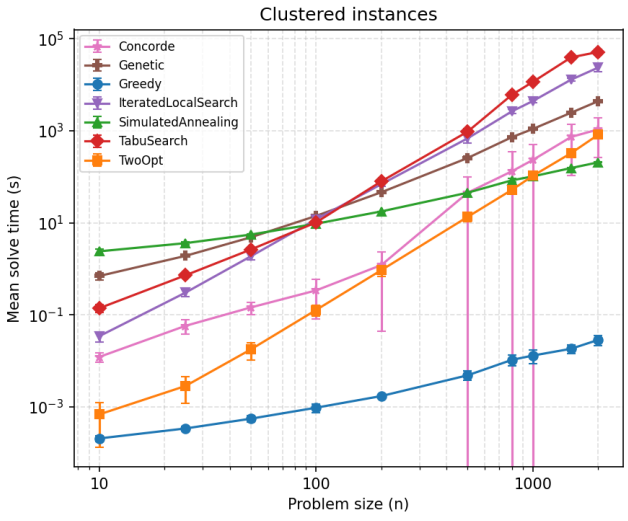

Fig. 3. Time scaling of clustered algorithms for different sizes and distributions.

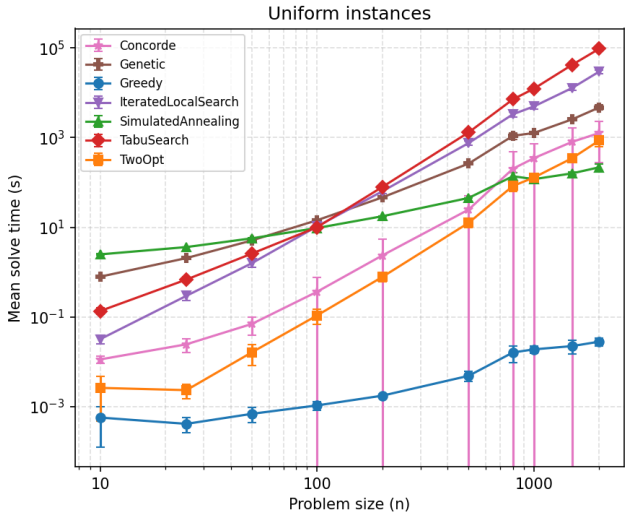

Fig. 4. Time scaling of uniformerd algorithms for different sizes and distributions.

VI. Discussion

Trade off analysis. Speed vs Quality.

Best all rounder, fast but sloppy, accurate but expensive...

Explain why the results look like that. Why is one algorithm better able to handle clusters than another. etc.

VII. Conclusion

We have presented a brief introduction to some of the most important methods to solve the Traveling Salesman Problem using heuristics and metaheuristics. After explaining the core concepts of these algorithms, we compared them using synthetic TSP instances, ranging in size from 10 to 2000 cities, and examined their scaling behaviour and their differences. This directly showcases them in a comparable way, providing new researchers in the TSP space with a good sense of how they perform. After reading this paper, it is our hope that the reader feels empowered to start their own research into TSP algorithms.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability*. wh freeman New York, 2002, vol. 29.

[2] J. Sui, S. Ding, X. Huang, Y. Yu, R. Liu, B. Xia, Z. Ding, L. Xu, H. Zhang, C. Yu, and D. Bu, "A survey on deep learning-based algorithms for the traveling salesman problem," *Frontiers of Computer Science*, vol. 19, no. 6, p. 196322, Jun. 2025.

[3] D. L. Applegate, R. E. Bixby, V. Chvátal, W. Cook, D. G. Espinoza, M. Goycoolea, and K. Helsgaun, "Certification of an optimal TSP tour through 85,900 cities," *Operations Research Letters*, vol. 37, no. 1, pp. 11–15, Jan. 2009.

[4] C. Rego and F. Glover, "Local search and metaheuristics," in *The Traveling Salesman Problem and Its Variations*, G. Gutin and A. P. Punnen, Eds. Boston, MA: Springer US, 2007, pp. 309–368.

[5] M. Gendreau and J.-Y. Potvin, "Tabu Search," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 37–55.

[6] H. R. Lourenço, O. C. Martin, and T. Stützle, "Iterated Local Search: Framework and Applications," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 129–168.

[7] P. Larrañaga, C. Kuijpers, R. Murga, I. Inza, and S. Dizdarevic, "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 129–170, Apr. 1999.
[8] D. Whitley, "Next Generation Genetic Algorithms: A User's Guide and Tutorial," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 245–274.
[9] M. Gendreau and J.-Y. Potvin, Eds., *Handbook of Metaheuristics*, ser. International Series in Operations Research & Management Science. Cham: Springer International Publishing, 2019, vol. 272.
[10] Krishnakumar R Iyer, "Tabu search algorithm for a thermal aware VLSI floorplanning application," 2013.
[11] I. H. Osman, "Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem," *Annals of Operations Research*, vol. 41, no. 4, pp. 421–451, Dec. 1993.

VIII. The First Appendix

A. Independence Declaration

if needed for the papper

We hereby declare that we have written this thesis independently as a group, that we have not used any sources or aids other than those indicated, and that we have not submitted this work, in whole or in part, for any other examination.

All content taken verbatim or in substance from published or unpublished sources has been clearly identified as such. Any use of AI-based tools has been disclosed in accordance with the institution's guidelines for scientific writing.

We have followed the rules set out in the Guidelines for Scientific Writing. We understand that any breach of this declaration may lead to the withdrawal of the qualifications or academic title awarded on the basis of this thesis.

Chur, June 6, 2026	Chur, June 6, 2026	Chur, June 6, 2026
_____	_____	_____
Signature	Signature	Signature

AI Assistance Declaration

Claude (Anthropic) and Codex were used during the writing of this report, mainly for translating technical content from German to English and occasionally as a sounding board when working through how to present an argument. Every output was checked and reworked by the authors before being included.